mSeva / DIGIT Architecture Guide

Payment Resettlement, Self-Healing & Reconciliation System

Production Ready | Kubernetes CronJob | Audit & Reporting

Abstract: An end-to-end technical manual and operational guide for resolving payment settlement discrepancies across Collection Services and Billing Services. Documents single on-demand resettlement, automated scheduled batch healing, database auditing, diagnostic SQL queries, and Kubernetes CronJob manifests.

Services:	`collection-services` , `billing-service`
Environments:	`Upgrade_mSeva` & `mseva` (mSevaUPYOG)
Database:	PostgreSQL (`egov_prod_db`)
Date:	October 2026

1. Executive Summary & Problem Statement

1.1 The Core Obstacle

In municipal tax systems (such as Water & Sewerage or Property Tax), bills carry strict expiration dates. When a citizen makes a payment against an **EXPIRED bill (Bill 1)**, the following failure sequence occurred:

> **Step 1 (Receipt Generation):** `collection-services` successfully persisted the payment and generated a receipt in `egcl_payment` referencing `Bill 1`.
>
> **Step 2 (Billing Status Lockout):** In `billing-service`, the query `BILL_STATUS_UPDATE_QUERY` was hardcoded with `WHERE status='ACTIVE'`. Because `Bill 1` was in `EXPIRED` status, 0 rows were updated, leaving `Bill 1` perpetually expired and demands uncollected.
>
> **Step 3 (Phantom Active Bill Trap):** When the citizen or system searched for bills again, `billing-service` generated a brand new **ACTIVE bill (Bill 2)**. If `Bill 1` was settled without cancelling `Bill 2`, `Bill 2` remained open and unpaid, prompting duplicate billing.

1.2 The Self-Healing Architecture

The solution addresses both the referential integrity of `egcl_paymentdetail.billid` (pointing to `Bill 1`) and cleans up any dangling phantom bills:

> **The 4-Step Healing Flow:**
>
> 1. **Phantom Cancellation:** Inspect `billing-service` for any newer active bill (`id != originalBillId`) and cancel it via `_cancelbill`.
> 2. **Original Bill Reactivation:** Transition `Bill 1` from `EXPIRED` to `ACTIVE` via `_reactivatebill`.
> 3. **Kafka Settlement:** Re-publish the payment event to Kafka topic `egov.collection.payment-create` so demands are settled and apportioned.
> 4. **Status Update to PAID:** Update `BILL_STATUS_UPDATE_QUERY` to allow both `ACTIVE` and `EXPIRED` bills to become `PAID`.
> 5. **Auditing:** Record run metrics in `egcl_resettle_batch` and granular payment records in `egcl_resettle_detail`.

2. Source Code Modifications

The following table details all modified and newly created components across both workspaces (`D:\Upgrade_mSeva\mSevaUPYOG` and `D:\mseva\mSevaUPYOG`):

Module	File Path	Role & Changes
Billing Service	`.../querybuilder/BillQueryBuilder.java`	Updated `BILL_STATUS_UPDATE_QUERY` : changed `WHERE status='ACTIVE'` to `WHERE (status='ACTIVE' OR status='EXPIRED')` .
Billing Service	`.../service/ReceiptServiceV2.java`	Added null-safety check for demands in `demandIdMap` (preventing NPEs) and explicitly invoked `billRepoV2.updateBillStatusBYId` to mark the bill `PAID` .
Billing Service	`.../service/BillServicev2.java` & `BillControllerv2.java`	Implemented and exposed endpoint `POST /bill/v2/_reactivatebill` .
Collection Services	`.../service/PaymentResettleService.java`	Implemented `resettlePayment` (single manual) and `autoResettlePayments` (batch/cron) with full safety buffers and error isolation.
Collection Services	`.../controller/PaymentController.java`	Exposed endpoints: `POST /payments/_resettle` , `POST /payments/_auto-resettle` , and `POST /payments/_resettle-audit` .
Collection Services	`.../repository/PaymentRepository.java`	Added queries to fetch candidate payments by time window, and insert/search batch and detail audit logs.
Collection Services	`.../resources/db/migration/main/V20261013130000...sql`	Flyway migration creating audit tables `egcl_resettle_batch` and `egcl_resettle_detail` with indexes.

3. Complete API Specifications

3.1 On-Demand Resettlement: `POST /payments/_resettle`

Used to immediately settle a specific stuck payment by connection number, receipt number, payment ID, or bill ID.

URL: `http://<host>:8280/collection-services/payments/_resettle`
Method: `POST`
Headers: `Content-Type: application/json`

Complete cURL Command

```
curl -X POST "http://localhost:8280/collection-services/payments/_resettle" \
  -H "Content-Type: application/json" \
  -d '{
    "RequestInfo": {
      "apiId": "org.egov.collection",
      "ver": "1.0",
      "ts": 1728988687000,
      "action": "_resettle",
      "did": "1",
      "key": "resettle-key",
      "msgId": "20261014160000|en_IN",
      "authToken": "YOUR_AUTH_TOKEN_HERE",
      "correlationId": "corr-uuid-12345",
      "userInfo": {
        "id": 1,
        "uuid": "40dceade-992d-4a8f-8243-19dda76a4171",
        "userName": "EMPLOYEE_ADMIN",
        "name": "System Admin",
        "mobileNumber": "9876543210",
        "emailId": "admin@punjab.gov.in",
        "type": "EMPLOYEE",
        "tenantId": "pb.amritsar",
        "roles": [
          { "id": 1, "name": "Super User", "code": "SUPERUSER", "tenantId": "pb" }
        ]
      }
    },
    "resettleCriteria": {
      "tenantId": "pb.amritsar",
      "consumerCode": "WS/107/2021-22/203456",
      "businessService": "WS",
      "receiptNumber": "07/2023-24/001234",
      "paymentId": "PAY-uuid-12345",
      "billId": "BILL-uuid-67890"
    }
  }'
```

Parameter Dictionary

Parameter	Type	Mandatory	Description
`RequestInfo`	Object	**Yes**	Authentication token and user identity header
`resettleCriteria.tenantId`	String	**Yes**	Target ULB (e.g., `"pb.amritsar"`)
`resettleCriteria.consumerCode`	String	Optional*	Water/Sewerage connection or Property ID
`resettleCriteria.businessService`	String	Optional*	Service code (`"WS"`, `"SW"`, `"PT"`)
`resettleCriteria.receiptNumber`	String	Optional*	Receipt number from `egcl_paymentdetail`
`resettleCriteria.paymentId`	String	Optional*	Payment UUID from `egcl_payment`
`resettleCriteria.billId`	String	Optional*	Bill UUID from `egbs_bill_v1`

*At least one identifier (consumerCode, receiptNumber, paymentId, or billId) must be provided.

3.2 Automated Batch Resettlement: `POST /payments/_auto-resettle`

Finds all successful payments within a target lookback window, checks if their bills are unpaid in billing service, and automatically heals them. Executed by the Kubernetes CronJob.

URL: `http://<host>:8280/collection-services/payments/_auto-resettle`
Method: `POST`
Headers: `Content-Type: application/json`

Complete cURL Command

```
curl -X POST "http://localhost:8280/collection-services/payments/_auto-resettle" \
  -H "Content-Type: application/json" \
  -d '{
    "RequestInfo": {
      "apiId": "org.egov.collection.cron",
      "ver": "1.0",
      "ts": 1728988687000,
      "action": "_auto-resettle",
      "did": "1",
      "key": "cron-key",
      "msgId": "cron-resettle-20261014",
      "authToken": "YOUR_AUTH_TOKEN_HERE",
      "userInfo": {
        "id": 1,
        "uuid": "cron-system-admin",
        "userName": "CRON_SUPERUSER",
        "type": "SYSTEM",
        "tenantId": "pb",
        "roles": [
          { "id": 1, "name": "Super User", "code": "SUPERUSER", "tenantId": "pb" }
        ]
      }
    },
    "criteria": {
      "tenantId": "pb",
      "businessService": "WS",
      "bufferMinutes": 30,
      "lookbackHours": 48,
      "limit": 50
    }
  }'
```

Batch Parameter Dictionary

Parameter	Type	Default	Description
`criteria.tenantId`	String	`null`	Specific ULB (`"pb.amritsar"`) or `"pb"` to scan all ULBs
`criteria.businessService`	String	`null`	Filter by service (e.g. `"WS"`) or omit to check all
`criteria.bufferMinutes`	Long	`30`	Excludes transactions in last 30m to avoid in-flight race conditions
`criteria.lookbackHours`	Long	`24`	Number of hours backwards from buffer to search
`criteria.limit`	Integer	`50`	Maximum payments to process per batch run (controls load)
`criteria.fromTime`	Long	Calculated	Explicit epoch ms timestamp (overrides lookbackHours)
`criteria.toTime`	Long	Calculated	Explicit epoch ms timestamp (overrides bufferMinutes)

3.3 Audit History Query: `POST /payments/_resettle-audit`

Enables administrators and developers to inspect recent batch runs and individual payment outcomes without direct SQL access.

URL: `http://<host>:8280/collection-services/payments/_resettle-audit?status=FAILED&limit=10`
Method: `POST`
Query Parameters: `batchId`, `paymentId`, `status` (SETTLED / SKIPPED / FAILED), `limit`

cURL Command

```
curl -X POST "http://localhost:8280/collection-services/payments/_resettle-audit?status=FAILED&limit=1
0" \
  -H "Content-Type: application/json" \
  -d '{
    "RequestInfo": {
      "apiId": "audit-query",
      "ver": "1.0",
      "authToken": "YOUR_AUTH_TOKEN_HERE",
      "userInfo": { "id": 1, "uuid": "admin-user" }
    }
  }'
```

--- # 4. Database Schema & Data Dictionary The migration file `V20261013130000__egcl_resettle_audit_tables_ddl.sql` creates two tables in PostgreSQL (`egov_prod_db`):

4.1 Table: `egcl_resettle_batch` (Execution Summaries)

Column Name	Data Type	Description
`id`	VARCHAR(64) PRIMARY KEY	Unique record ID (UUID)
`batch_id`	VARCHAR(64)	Batch UUID (or `MANUAL-xxxx` for manual calls)
`tenantid`	VARCHAR(256)	Tenant filter used
`businessservice`	VARCHAR(256)	Business service filter used
`from_time`	BIGINT	Start epoch timestamp of scanned payments
`to_time`	BIGINT	End epoch timestamp of scanned payments
`total_found`	INTEGER	Candidate payments found in time window
`total_settled`	INTEGER	Count of payments successfully healed
`total_skipped`	INTEGER	Count of payments skipped (already PAID)
`total_failed`	INTEGER	Count of payments that encountered an error
`status`	VARCHAR(32)	`COMPLETED`, `PARTIAL`, or `FAILED`
`createdtime`	BIGINT	Timestamp when the job executed
`createdby`	VARCHAR(64)	User UUID who ran the job

4.2 Table: `egcl_resettle_detail` (Granular Payment Records & Errors)

Column Name	Data Type	Description
`id`	VARCHAR(64) PRIMARY KEY	Unique record ID (UUID)
`batch_id`	VARCHAR(64)	Links to `egcl_resettle_batch.batch_id`
`paymentid`	VARCHAR(64) NOT NULL	Payment ID from `egcl_payment`
`consumercode`	VARCHAR(256)	Connection number or Property ID
`businessservice`	VARCHAR(256)	Business service (e.g. `WS`)
`billid`	VARCHAR(64)	Original bill ID from `egbs_bill_v1`
`tenantid`	VARCHAR(256)	ULB tenant ID
`resettle_status`	VARCHAR(32)	`SETTLED`, `SKIPPED`, or `FAILED`
`failure_reason`	TEXT	Exact error message or success confirmation
`createdtime`	BIGINT	Timestamp of execution
`createdby`	VARCHAR(64)	User UUID

5. Production SQL Playbook

5.1 Diagnostic Query: Find Connections with Receipts Against Unsettled Bills

Run this query to locate real production cases where payment was deposited but demands are not collected:

```
SELECT
    COALESCE(b.consumercode, clb.consumercode) AS connection_number,
    pyd.businessservice,
    py.tenantid,
    pyd.receiptnumber,
    py.transactionnumber,
    py.totalamountpaid AS receipt_amount,
    to_timestamp(py.createdtime / 1000) AS receipt_date,
    pyd.billid AS bill_id,
    b.status AS bill_status_in_billing,
    COALESCE(SUM(dd.taxamount), 0) AS total_tax_demanded,
    COALESCE(SUM(dd.collectionamount), 0) AS total_collected_amount,
    (COALESCE(SUM(dd.taxamount), 0) - COALESCE(SUM(dd.collectionamount), 0)) AS pending_demand_balance,
    py.id AS payment_id
FROM egcl_payment py
INNER JOIN egcl_paymentdetail pyd ON pyd.paymentid = py.id
INNER JOIN egbs_bill_v1 b ON b.id = pyd.billid
LEFT JOIN egcl_bill clb ON clb.id = pyd.billid
LEFT JOIN egbs_billdetail_v1 bd ON bd.billid = b.id
LEFT JOIN egbs_demand_v1 d ON d.id = bd.demandid
LEFT JOIN egbs_demanddetail_v1 dd ON dd.demandid = d.id
WHERE py.paymentstatus IN ('DEPOSITED', 'NEW')
  AND pyd.receiptnumber IS NOT NULL
  AND (b.status != 'PAID' OR d.ispaymentcompleted = FALSE)
  AND pyd.businessservice IN ('WS', 'SW', 'WS.ONE_TIME_FEE', 'SW.ONE_TIME_FEE')
GROUP BY
    b.consumercode,
    clb.consumercode,
    pyd.businessservice,
    py.tenantid,
    pyd.receiptnumber,
    py.transactionnumber,
    py.totalamountpaid,
    py.createdtime,
    pyd.billid,
    b.status,
    py.id
HAVING
    (COALESCE(SUM(dd.taxamount), 0) > COALESCE(SUM(dd.collectionamount), 0))
    OR b.status IN ('EXPIRED', 'ACTIVE')
ORDER BY py.createdtime DESC
LIMIT 5;
```

5.2 Fast Query: Receipts where Bill is `EXPIRED`

```
SELECT
    COALESCE(b.consumercode, clb.consumercode) AS connection_number,
    pyd.businessservice,
    py.tenantid,
    pyd.receiptnumber,
    py.totalamountpaid AS receipt_amount,
    to_timestamp(py.createdtime / 1000) AS receipt_date,
    pyd.billid,
    b.status AS bill_status_in_billing,
    py.id AS payment_id
FROM egcl_payment py
INNER JOIN egcl_paymentdetail pyd ON pyd.paymentid = py.id
INNER JOIN egbs_bill_v1 b ON b.id = pyd.billid
LEFT JOIN egcl_bill clb ON clb.id = pyd.billid
WHERE py.paymentstatus IN ('DEPOSITED', 'NEW')
  AND pyd.receiptnumber IS NOT NULL
  AND b.status = 'EXPIRED'
ORDER BY py.createdtime DESC
LIMIT 5;
```

5.3 Inspect Audit History in Database

```
-- 1. Check recent execution batch runs
SELECT batch_id, total_found, total_settled, total_skipped, total_failed, status, to_timestamp(createdt
ime/1000) as run_time
FROM egcl_resettle_batch
ORDER BY createdtime DESC LIMIT 10;

-- 2. Check failed settlements and error reasons
SELECT paymentid, consumercode, businessservice, failure_reason, to_timestamp(createdtime/1000) as sett
led_time
FROM egcl_resettle_detail
WHERE resettle_status = 'FAILED'
ORDER BY createdtime DESC LIMIT 10;
```

6. Kubernetes CronJob Operations

6.1 Manifest Specification (`collection-resettle-cronjob.yaml`)

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: collection-resettle-job
  namespace: egov
  labels:
    app: collection-resettle-job
    group: billing-collection
spec:
  schedule: "0 */2 * * *"         # Runs every 2 hours
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 5
  concurrencyPolicy: Forbid       # Prevents overlapping runs
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        metadata:
          labels:
            app: collection-resettle-job
        spec:
          restartPolicy: OnFailure
          containers:
          - name: resettle-trigger
            image: curlimages/curl:7.85.0
            imagePullPolicy: IfNotPresent
            command:
            - /bin/sh
            - -c
            - |
              echo "Starting collection resettle batch job at $(date)..."
              PAYLOAD='{
                "RequestInfo": {
                  "apiId": "collection-resettle-cron",
                  "ver": "1.0",
                  "action": "_auto-resettle",
                  "did": "1",
                  "key": "cron",
                  "msgId": "cron-resettle-request",
                  "userInfo": {
                    "id": 1,
                    "uuid": "cron-system-admin",
                    "type": "SYSTEM",
                    "roles": [{"name": "SUPERUSER", "code": "SUPERUSER"}]
                  }
                },
                "criteria": {
                  "limit": 50,
```

```
                  "bufferMinutes": 30,
                  "lookbackHours": 24
                }
              }'

              RESPONSE=$(curl -s -w "\nHTTP_STATUS:%{http_code}" -X POST \
                -H "Content-Type: application/json" \
                -d "$PAYLOAD" \
                "http://collection-services.egov.svc.cluster.local:8080/collection-services/payments/_a
uto-resettle")

              HTTP_BODY=$(echo "$RESPONSE" | sed -e '$d')
              HTTP_STATUS=$(echo "$RESPONSE" | tail -n 1 | cut -d':' -f2)

              echo "Response Code: $HTTP_STATUS"
              echo "Response Body: $HTTP_BODY"

              if [ "$HTTP_STATUS" -ne 200 ]; then
                echo "ERROR: Resettle job failed with HTTP status $HTTP_STATUS"
                exit 1
              fi
              echo "Resettle batch job completed successfully at $(date)."
```

6.2 Operational Commands

Action	Command Line
Deploy CronJob	`kubectl apply -f collection-resettle-cronjob.yaml`
View Status	`kubectl get cronjob collection-resettle-job -n egov`
Trigger Immediate Manual Run	`kubectl create job --from=cronjob/collection-resettle-job test-run-01 -n egov`
View Execution Logs	`kubectl logs -l job-name=test-run-01 -n egov`

> **Deployment Note:** Remember to restart or rolling-update the `collection-services` deployment pods (`kubectl rollout restart deployment/collection-services -n egov`) so the running pods load the newly compiled classes and audit persistence routines.